

离散化

人员

周子一、杨瑾硕、秦显森、于子珈、牛同泽、陈洛冉、司云心、牛同泽 到课, 李子瀚 线上

上周作业检查

上周作业链接: <https://cppoj.kids123code.com/contest/4812>

王向东老师周日十点半C++hash							
刷新							
#	用户名	姓名	编程分	时间	A	B	C
1	yuzijia1	于子珈	300	998	100	100	100
2	zhouziyi	周子一	200	373	100	100	
3	siyunxin	司云心	200	832	100	100	
4	lizihan	李子瀚	200	986	100	100	
5	niutongze	牛同泽	200	998	100	100	
6	xieyakai	谢亚楷	200	1045	100	100	
7	chenluoran	陈洛冉	200	1100	100	100	
8	yangjinshuo	杨瑾硕	100	36	100		
9	qinxiansen	秦显森	100	163	100	0	

本周作业

<https://cppoj.kids123code.com/contest/4952> (课上讲了 A ~ D 题, 课后作业是 C D 题必做)

课堂表现

今天讲了 离散化 这个内容, 离散化 同学们都掌握的比较好, 但课上反映出来的问题是同学们 树状数组 不熟, 建议同学们课下再复习一下之前的 树状数组。

课堂内容

区间和

离散化 + 前缀和 即可

```
#include <bits/stdc++.h>

using namespace std;

typedef long long LL;
const int maxn = 3e5 + 5;
int x[maxn], c[maxn];
int l[maxn], r[maxn];
LL w[maxn], p[maxn];
```

```

vector<int> ys;
int fFind(int x) {
    return lower_bound(ys.begin(), ys.end(), x) - ys.begin() + 1;
}

int main()
{
    int n, m; cin >> n >> m;
    for (int i = 1; i <= n; ++i) cin >> x[i] >> c[i], ys.push_back(x[i]);
    for (int i = 1; i <= m; ++i) {
        cin >> l[i] >> r[i]; ys.push_back(l[i]), ys.push_back(r[i]);
    }

    sort(ys.begin(), ys.end());
    ys.erase(unique(ys.begin(), ys.end()), ys.end());

    for (int i = 1; i <= n; ++i) {
        int pos = fFind(x[i]); w[pos] += c[i];
    }
    for (int i = 1; i < maxn; ++i) p[i] = p[i-1] + w[i];

    for (int i = 1; i <= m; ++i) {
        int ll = fFind(l[i]), rr = fFind(r[i]);
        cout << p[rr] - p[ll-1] << endl;
    }
    return 0;
}

```

逆序对

离散化 + 树状数组

查找每个数前面有多少比他大的

```

#include <bits/stdc++.h>

using namespace std;

vector<int> ys;
int yFind(int x) { return lower_bound(ys.begin(), ys.end(), x) - ys.begin(); }

typedef long long LL;
const int maxn = 5e5 + 5;
int w[maxn];

int tr[maxn];
int lowbit(int x) { return x & (-x); }
void update(int x, int k) {
    while (x < maxn) tr[x] += k, x += lowbit(x);
}
int query(int x) {

```

```

int res = 0;
while (x) res += tr[x], x -= lowbit(x);
return res;
}

int main()
{
    int n; cin >> n;
    for (int i = 1; i <= n; ++i) cin >> w[i], ys.push_back(w[i]);
    sort(ys.begin(), ys.end()), ys.erase(unique(ys.begin(), ys.end()), ys.end());

    LL res = 0;
    for (int i = 1; i <= n; ++i) {
        int x = yFind(w[i])+1;
        res += (i-1) - query(x);
        update(x, 1);
    }
    cout << res << endl;
    return 0;
}

```

中位数

离散化 + 二分 + 树状数组

二分 + 树状数组 是很常用的方式

```

#include <bits/stdc++.h>

using namespace std;

vector<int> ys;
int yFind(int x) { return lower_bound(ys.begin(),ys.end(),x) - ys.begin() + 1; }

const int maxn = 1e5 + 5;
int w[maxn], tr[maxn];

int lowbit(int x) { return x&(-x); }
void update(int x, int k) {
    while (x < maxn) tr[x] += k, x += lowbit(x);
}
int query(int x) {
    int res = 0;
    while (x) res += tr[x], x -= lowbit(x);
    return res;
}

int main()
{
    int n; cin >> n;
    for (int i = 1; i <= n; ++i) {

```

```

        cin >> w[i]; ys.push_back(w[i]);
    }
    sort(ys.begin(), ys.end()); ys.erase(unique(ys.begin(),ys.end()),ys.end());

    for (int i = 1; i <= n; ++i) {
        int x = yFind(w[i]); update(x,1);
        if (i & 1) {
            int l = 1, r = maxn-1;
            while (l <= r) {
                int mid = (l + r) / 2;
                if (query(mid) >= (i+1)/2) r = mid-1;
                else l = mid+1;
            }
            cout << ys[l-1] << endl;
        }
    }
    return 0;
}

```

[USACO06DEC] Milk Patterns G

二分长度 mid, 记录每个长度为 mid 的子串的 hash 值, 判断有没有重复出现 K 次的

```

#include <bits/stdc++.h>

using namespace std;

typedef unsigned ULL;
const int maxn = 20000 + 5;
const int P = 1e9+7;
int w[maxn];
ULL p[maxn], h[maxn];

ULL get_hash(int l, int r) { return h[r] - h[l-1]*p[r-l+1]; }

bool check(int n, int K, int mid) {
    map<ULL, int> mp;
    for (int i = 1, j = mid; j <= n; ++i, ++j) mp[get_hash(i,j)]++;

    for (auto it : mp) {
        if (it.second >= K) return true;
    }
    return false;
}

int main()
{
    int n, K; cin >> n >> K;
    for (int i = 1; i <= n; ++i) cin >> w[i];

    p[0] = h[0] = 1;

```

```
for (int i = 1; i <= n; ++i) {  
    p[i] = p[i-1] * P, h[i] = h[i-1] * P + w[i];  
}  
  
int l = 1, r = n;  
while (l <= r) {  
    int mid = (l + r) / 2;  
    if (check(n, K, mid)) l = mid+1;  
    else r = mid-1;  
}  
cout << r << endl;  
return 0;  
}
```